

AppSec Review & Threat Modeling in SDL

The review loop (Microsoft SDL style)

Requirements – security requirements per feature (authz scope, data classification)

Design – threat model the feature before code

Implementation – secure coding standards, SAST in CI

Verification – code review, dynamic tests, fuzzing

Response – incident plan and post-release monitoring

Practical review checklist (web)

Authentication: brute-force protection, session fixation, logout

Authorization: server-side checks per object (IDOR!)

Input: validation, length, encoding on output

Secrets: none in code, none in logs

Sessions: HttpOnly+Secure flags, rotation on privilege change

Crypto: modern ciphersets, no homegrown crypto

Headers: CSP, HSTS, X-Frame-Options, Referrer-Policy

Dependencies: SCA coverage, no known-critical CVEs

Rate limits: login, forms, APIs

Logging: authz events, anomalies, no secrets in logs

Writing findings devs love (and fix)

Good finding:

Title: "IDOR allows order enumeration for any user"

Severity & CVSS: 8.1 (High)

Affected: GET /api/orders/{id}, no ownership check

Repro: curl with other user's id

Impact: disclosure of order PII

Fix: verify resource ownership server-side before returning

Verify, don't assume

Run the test yourself (legal scope!), capture evidence (request/response, redacted), and only report what you reproduced. Then track the fix to done, then re-test.

A review that names a risk without a reproducible repro wastes everyone's time. Evidence-first reporting is the difference between a "ticket" and a "security fix."

[← CI/CD Security & Software Su...](#)

Module complete ✓